WINDOWS REGISTRY CHANGE MONITORING SYSTEM

Practical Work Documentation

Student Name:

Vaishali Vasant Kadam

Project Title:

Windows Registry Change Monitoring System

Submitted For:

Internship Practical Documentation

GitHub:

https://github.com/kadamvaishali378/Windows-Registry-Change-Monitoring-System

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the organization and mentors who provided me with the opportunity to complete my Cyber Security Internship and work on the project titled **“Windows Registry Change Monitoring System.”**

This project allowed me to gain practical exposure to system-level security monitoring and understand how registry-based persistence mechanisms are used by malware.

I would also like to thank my internship mentor and the organization for providing valuable guidance, resources, and support throughout the development of this project. Their guidance helped me understand the practical implementation of defensive cybersecurity techniques such as registry monitoring, persistence detection, and security policy analysis.

I am also grateful for the opportunity to apply theoretical cybersecurity concepts in a practical environment. Working on this project helped strengthen my knowledge of Windows system internals, Python scripting, and security monitoring techniques used in real-world environments.

Finally, I would like to thank everyone who contributed directly or indirectly to the successful completion of this project.

DECLARATION

I hereby declare that the project titled **“Windows Registry Change Monitoring System”** submitted as part of my **Cyber Security Internship** is my original work.

This project has been developed by me during my internship period for the purpose of learning and demonstrating practical cybersecurity monitoring techniques. The implementation, analysis, and documentation presented in this report are based on the work carried out by me.

All references to external materials, documentation, and technical resources have been appropriately acknowledged where applicable.

I confirm that this project has not been submitted previously for any academic or professional evaluation.

Student Name: **Vaishali Vasant Kadam**

Contents

1. PROJECT OVERVIEW / DESCRIPTION 6
2. PRACTICAL MOTIVATION 7
3. PROJECT OBJECTIVES 8
4. PRACTICAL SCOPE OF THE PROJECT 10
5. TOOLS & TECHNOLOGIES USED 13
6. SYSTEM IMPLEMENTATION 14
7. WORKFLOW / SYSTEM ARCHITECTURE 17
8. SYSTEM ARCHITECTURE 18
9. FLOWCHART 21
10. EXPECTED PRACTICAL OUTPUT 22
11. LEARNING OUTCOMES 26
12. ADVANTAGES OF THE SYSTEM 27
13. LIMITATIONS OF THE CURRENT SYSTEM 28
14. FUTURE SCOPE 30
15. CONCLUSION 31
16. PROJECT DELIVERABLES 32

Table of Figures

Figure 1: baseline.json 11
Figure 2: monitor.py code 15
Figure 3: Architecture Diagram of Windows Registry Change Monitoring System 19
Figure 4: Flowchart 21
Figure 5: logs.txt 22
Figure 6: monitor.py running in terminal 23
Figure 7: Suspicious Defender policy detected 23
Figure 8: Registry Editor 24
Figure 9: logs.txt 24
Figure 10: baseline.json 25

1. PROJECT OVERVIEW / DESCRIPTION

The Windows Registry is a hierarchical database used by the Windows operating system to store configuration settings for the operating system, installed applications, hardware components, and security policies. It contains critical information related to system startup behavior, software configuration, and system security settings.

Because of its importance, the registry is frequently targeted by malware and attackers. Malicious software often modifies registry keys to establish persistence, disable security protections, manipulate system behavior, or automatically execute malicious programs during system startup.

This project implements a **Windows Registry Change Monitoring System** that detects suspicious or unauthorized changes made to selected registry locations. The monitoring system focuses on identifying two common attack techniques:

- Malware persistence through **autorun registry entries**
- Security policy tampering such as **disabling Windows Defender**

The monitoring tool is implemented using **Python** and the **winreg module**, which allows programs to read and interact with the Windows Registry.

The system performs the following security checks:

- Reads the current registry values of the **autorun registry key**
- Compares these values with a previously created **baseline registry snapshot**
- Detects newly added startup entries that may indicate persistence mechanisms
- Checks Windows Defender policy settings to detect potential **security tool tampering**

When suspicious registry changes are detected, the system generates alerts and records the events in a log file with timestamps. These logs can be used for security monitoring, malware investigation, or digital forensic analysis.

The project demonstrates how basic registry monitoring techniques can be implemented to detect persistence mechanisms and configuration tampering in Windows systems.

2. PRACTICAL MOTIVATION

The Windows Registry plays a central role in system configuration and application behavior. Because registry values control important system operations, attackers often modify registry entries to maintain unauthorized access or bypass security controls.

One common technique used by malware is the creation of **startup persistence mechanisms**. By adding entries to autorun registry keys, malicious programs can automatically execute whenever a user logs into the system.

Another attack technique involves modifying registry policy settings to disable security protections such as **Windows Defender**. Malware may attempt to disable antivirus software to avoid detection.

Common malicious activities involving registry manipulation include:

- Persistent malware installation
- Ransomware configuration changes
- Unauthorized startup program entries
- Privilege escalation attempts
- Security tool tampering

Detecting such unauthorized registry modifications is an important task in cybersecurity monitoring and incident response.

This project simulates a **basic defensive monitoring system** that checks important registry locations for suspicious changes. The implemented system demonstrates how defenders can monitor registry activity and detect potential indicators of compromise.

The knowledge gained from this project is relevant to real-world cybersecurity roles such as:

- Security Operations Center (SOC) monitoring
- Incident Response investigations
- Malware analysis
- Digital forensic analysis

3. PROJECT OBJECTIVES

The primary objective of this project is to design and implement a basic **Windows Registry Monitoring System** capable of detecting suspicious configuration changes that may indicate malware activity or unauthorized system modifications.

Since the Windows Registry stores critical system and application settings, monitoring sensitive registry locations can help detect persistence mechanisms and security policy tampering commonly used by malicious software.

The specific objectives of this project include the following:

1. Monitor the Autorun Registry Key

The system monitors the autorun registry key located at:

HKCU\Software\Microsoft\Windows\CurrentVersion\Run

This registry key contains entries for programs that automatically start when a user logs into the system. Malware often abuses this location to establish persistence.

The monitoring script reads the registry values and detects any new startup entries that were not present in the baseline configuration.

2. Detect Potential Malware Behavior Through Security Policy Monitoring

The system also checks security policy registry keys to identify possible attempts to disable system protections.

The monitoring script inspects the registry path:

HKLM\Software\Policies\Microsoft\Windows Defender

Specifically, it checks the value **DisableAntiSpyware**. If this value indicates that Windows Defender has been disabled, the system generates a security alert.

3. Implement a Baseline Registry Snapshot

To detect registry changes, the system first creates a **baseline snapshot** stored in a file named **baseline.json**.

This file records the original registry entries present in the monitored autorun key and serves as a trusted reference for later comparisons.

4. Compare Current Registry Values with the Baseline

The monitoring script retrieves the current registry values and compares them with the baseline snapshot.

If new entries appear that were not present in the baseline configuration, they are flagged as potential unauthorized modifications.

5. Generate Alerts and Record Suspicious Changes

Whenever suspicious registry activity is detected, the system generates alerts and records the event in a log file named **logs.txt**.

Each log entry includes the timestamp, type of alert, and description of the detected registry change, allowing security analysts to review and investigate potential security incidents.

4. PRACTICAL SCOPE OF THE PROJECT

The registry monitoring system focuses on detecting suspicious changes in registry locations commonly used by malware.

The monitoring functionality is divided into several modules.

4.1 Autorun Key Monitoring

Malware frequently establishes persistence by adding entries to autorun registry keys. These entries cause programs to automatically start whenever the user logs into the system.

The implemented system monitors the registry key:

HKCU\Software\Microsoft\Windows\CurrentVersion\Run

This key contains programs configured to automatically execute at user login.

The monitoring script performs the following operations:

- Reads current registry values using the Python winreg module
- Compares the retrieved values with a previously stored baseline snapshot
- Detects newly added registry entries that were not present in the baseline

If a new startup entry is detected, the system generates an alert and records the event in the monitoring log.

4.2 Malware-Like Change Detection

Malware may attempt to disable security protections such as Windows Defender by modifying registry policy settings.

The monitoring system checks the registry path:

HKLM\Software\Policies\Microsoft\Windows Defender

Within this key, the script checks the value:

DisableAntiSpyware

If this value is set to **1**, it may indicate that Windows Defender has been disabled through registry policy modification.

The system reads this registry value and generates a security alert if suspicious behavior is detected.

4.3 Registry Integrity Checker

The project includes a **baseline registry snapshot mechanism** to detect unexpected changes.

A baseline file named **baseline.json** stores the initial state of the monitored autorun registry key. This file contains:

- Timestamp of baseline creation
- List of registry entries present during baseline capture

During monitoring, the script loads this baseline file and compares it with the current registry values.

If a new entry appears that was not part of the baseline snapshot, it is treated as a potential persistence mechanism and an alert is generated.

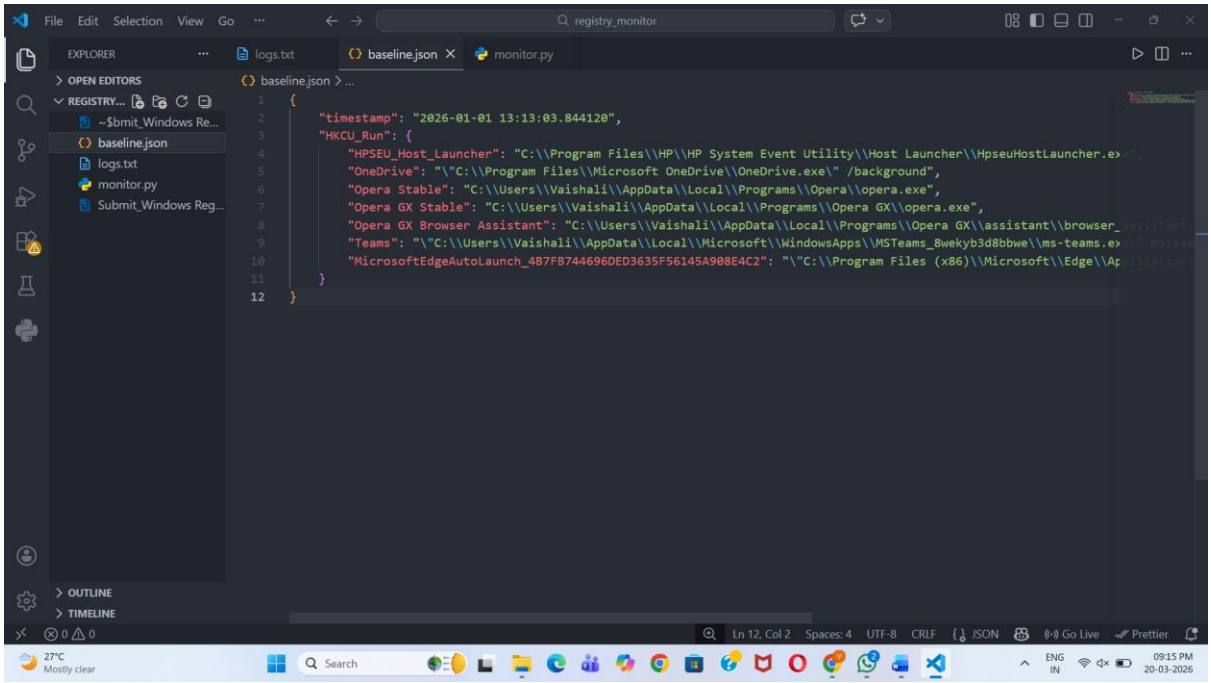

Figure 1: baseline.json

4.4 Alerting and Reporting Module

The monitoring system includes a logging mechanism that records detected events.

Whenever a suspicious registry change is identified, the system:

- Generates an alert message
- Displays the alert in the terminal
- Writes the alert to a log file

The log file used in this project is:

logs.txt

Each log entry contains:

- Timestamp
- Alert category
- Description of the detected registry change

These logs allow security analysts to review registry activity and identify possible system compromise.

5. TOOLS & TECHNOLOGIES USED

The development and documentation of the **Windows Registry Change Monitoring System** required the use of several programming tools, libraries, and software components. Each tool played a specific role in implementing the monitoring functionality and documenting the project.

Tool / Technology	**Purpose**
Python	Python is the primary programming language used to implement the registry monitoring system. It was chosen because it provides powerful libraries for interacting with operating system components and allows rapid development of security monitoring scripts.
winreg	The winreg module is a built-in Python library used to access and read values from the Windows Registry. It enables the script to open registry keys, retrieve stored values, and analyze registry entries.
json	The json module is used to store and load the baseline registry snapshot in a structured JSON file. This file acts as a reference for comparing the current registry state with the original baseline configuration.
datetime	The datetime module is used to generate timestamps for log entries. This helps record when registry changes were detected, which is useful for monitoring and forensic analysis.
Windows Operating System	The project is designed specifically for the Windows environment because it interacts directly with the Windows Registry, which stores system configuration and application settings.
Microsoft Word	Microsoft Word was used to prepare the project documentation, including formatting the report, creating tables, and organizing the project sections in a structured manner.

6. SYSTEM IMPLEMENTATION

The Windows Registry Change Monitoring System is implemented as a Python-based monitoring script named **monitor.py**.
This script interacts directly with the Windows Registry using the built-in **winreg** module.

The purpose of the script is to read registry values from selected registry paths, compare them with a previously stored baseline configuration, and generate alerts if suspicious changes are detected.

The system focuses on monitoring registry locations commonly used for malware persistence and security policy modifications.

The script is divided into multiple functions, each responsible for performing a specific monitoring task.

6.1 get_current_run_key()

This function retrieves the current registry values stored in the **autorun registry key**:

HKCU\Software\Microsoft\Windows\CurrentVersion\Run

This registry location contains programs configured to automatically execute when a user logs into the system.

The function performs the following operations:

- Opens the registry key using the **winreg.OpenKey()** function
- Enumerates all registry values using **winreg.EnumValue()**
- Stores the retrieved registry entries in a Python dictionary

The collected data represents the **current state of the autorun registry key**, which is later used for comparison with the baseline snapshot.

6.2 check_autorun_changes()

This function detects suspicious startup entries by comparing the **current registry values** with the **baseline registry snapshot** stored in the file **baseline.json**.

The function performs the following steps:

1. Loads the baseline registry data from **baseline.json**
2. Retrieves the current autorun registry entries using the get_current_run_key() function
3. Compares the baseline entries with the current registry values

4. Identifies new entries that were not present during baseline creation

If a new registry entry is detected, it may indicate that a program has been added to run automatically at system startup.

When such a change is detected, the system generates a **security alert** and records the event in the monitoring log file.

6.3 check_defender_tampering()

This function checks for possible attempts to disable **Windows Defender** through registry policy modifications.

The function reads the registry path:

HKLM\Software\Policies\Microsoft\Windows Defender

Within this location, the script checks the value:

DisableAntiSpyware

If this value is set to **1**, it indicates that Windows Defender may have been disabled through a registry policy change.

Disabling security tools is a common technique used by malware to avoid detection.

If this condition is detected, the monitoring system generates a **security alert** and records the event in the log file for further analysis.

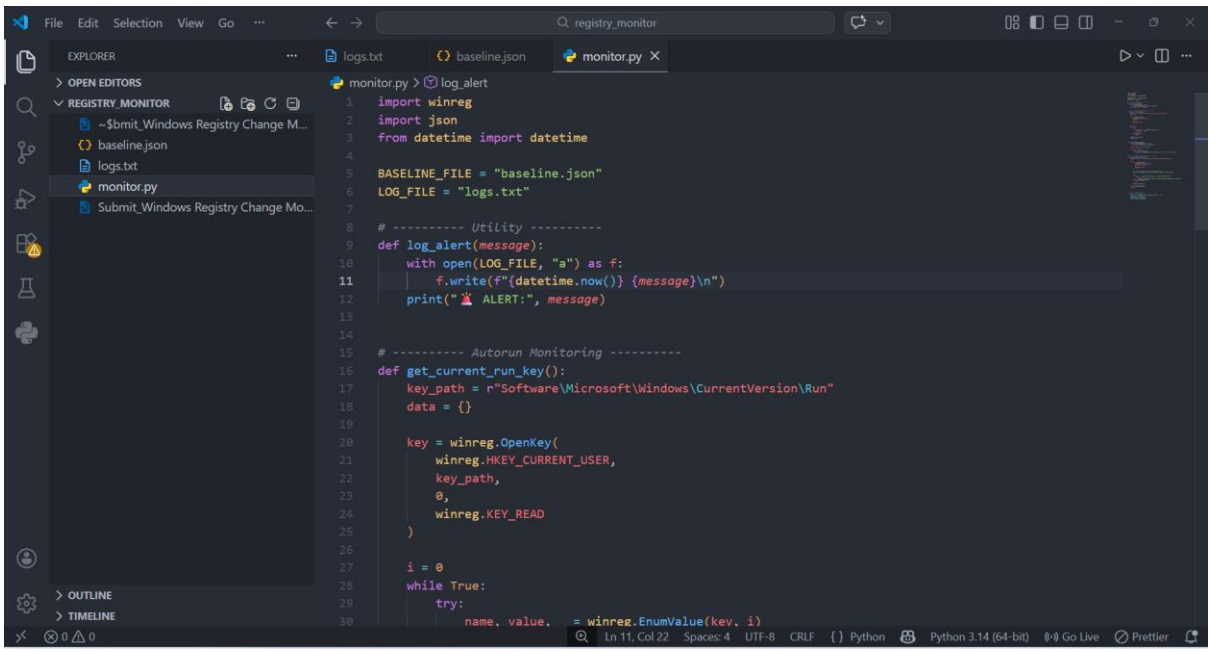

Figure 2: monitor.py code

```python
def get_current_run_key():
    return data


def check_autorun_changes():
    with open(BASELINE_FILE, "r") as f:
        baseline = json.load(f)["HKCU_Run"]

    current = get_current_run_key()

    for key in current:
        if key not in baseline:
            log_alert(f"[AUTORUN] New startup entry: {key} -> {current[key]}")


# ---------- Malware-Like Detection ----------
def check_defender_tampering():
    defender_key_path = r"Software\Policies\Microsoft\Windows Defender"

    try:
        key = winreg.OpenKey(
            winreg.HKEY_LOCAL_MACHINE,
            defender_key_path,
            0,
            winreg.KEY_READ
        )

        # 🔴 TEMP TEST ALERT (SAFE — REMOVE AFTER TEST)
        log_alert("[SECURITY] Suspicious Defender policy detected (TEST)")
```

```python
def get_current_run_key():

    key = winreg.OpenKey(
        winreg.HKEY_CURRENT_USER,
        key_path,
        0,
        winreg.KEY_READ
    )

    i = 0
    while True:
        try:
            name, value, _ = winreg.EnumValue(key, i)
            data[name] = value
            i += 1
        except OSError:
            break

    winreg.CloseKey(key)
    return data


def check_autorun_changes():
    with open(BASELINE_FILE, "r") as f:
        baseline = json.load(f)["HKCU_Run"]

    current = get_current_run_key()

    for key in current:
        if key not in baseline:
```

```python

# ---------- Malware-Like Detection ----------
def check_defender_tampering():
    defender_key_path = r"Software\Policies\Microsoft\Windows Defender"

    try:
        key = winreg.OpenKey(
            winreg.HKEY_LOCAL_MACHINE,
            defender_key_path,
            0,
            winreg.KEY_READ
        )

        # 🔴 TEMP TEST ALERT (SAFE — REMOVE AFTER TEST)
        log_alert("[SECURITY] Suspicious Defender policy detected (TEST)")

        try:
            value, _ = winreg.QueryValueEx(key, "DisableAntiSpyware")
            if value == 1:
                log_alert("[SECURITY] Windows Defender appears DISABLED via registry")
        except FileNotFoundError:
            pass

        winreg.CloseKey(key)

    except FileNotFoundError:
        pass
```

7. WORKFLOW / SYSTEM ARCHITECTURE

The Windows Registry Change Monitoring System follows a structured workflow designed to detect suspicious registry modifications and generate security alerts. The workflow represents the sequence of steps performed by the monitoring script during execution.

The monitoring process begins by identifying sensitive registry locations that are commonly targeted by malware for persistence or security policy manipulation. These registry keys are selected as monitoring points.

Step 1 – Define Sensitive Registry Keys

The system defines the registry paths that will be monitored for suspicious changes. In this project, the main keys monitored include the autorun registry key and Windows Defender policy registry settings.

Step 2 – Load Baseline Registry Snapshot

The script loads the baseline registry snapshot stored in the **baseline.json** file. This file contains the original registry values captured during baseline creation and represents the trusted system configuration.

Step 3 – Read Current Registry Values

Using the **winreg** module, the script accesses the Windows Registry and retrieves the current values stored in the monitored registry keys.

Step 4 – Compare Registry Values with Baseline

The current registry values are compared with the baseline snapshot to identify any differences between the trusted configuration and the current system state.

Step 5 – Detect Newly Added Autorun Entries

If a new registry entry appears in the autorun registry key that was not present in the baseline snapshot, it is flagged as a suspicious startup entry. Such entries may indicate a potential persistence mechanism used by malware.

Step 6 – Check Windows Defender Policy Settings

The system checks the Windows Defender policy registry key to determine whether security protections have been disabled. If the **DisableAntiSpyware** value indicates that Defender has been disabled, a security alert is generated.

Step 7 – Generate Alerts and Monitoring Logs

Whenever suspicious registry activity is detected, the system generates an alert and records the event in the monitoring log file **logs.txt** along with a timestamp. These logs provide a record of detected registry changes for further analysis.

8. SYSTEM ARCHITECTURE

The Windows Registry Change Monitoring System follows a simple monitoring architecture designed to detect suspicious registry modifications and generate security alerts.

The system is implemented as a Python-based monitoring script that interacts directly with the Windows Registry using the **winreg module**. The architecture focuses on monitoring specific registry locations that are commonly used for malware persistence and security policy modifications.

The system performs registry scans, compares the results with a baseline configuration, and generates alerts if suspicious changes are detected.

Components of the Architecture

1. Registry Monitoring Module

This component reads registry values from selected registry paths using the Python winreg module. It retrieves the current state of registry entries from the monitored keys.

2. Baseline Snapshot Module

The baseline module stores the initial registry state in a JSON file named **baseline.json**. This snapshot represents the trusted configuration used for comparison during monitoring.

3. Change Detection Engine

This component compares the current registry values with the baseline configuration. If a new registry entry appears that was not present in the baseline snapshot, it is flagged as a suspicious change.

4. Security Policy Checker

This module checks registry keys associated with Windows Defender policies to detect possible attempts to disable security protections.

5. Alert and Logging Module

When suspicious activity is detected, the system generates alerts and records the event in the monitoring log file **logs.txt** with timestamps.

Architecture Diagram:

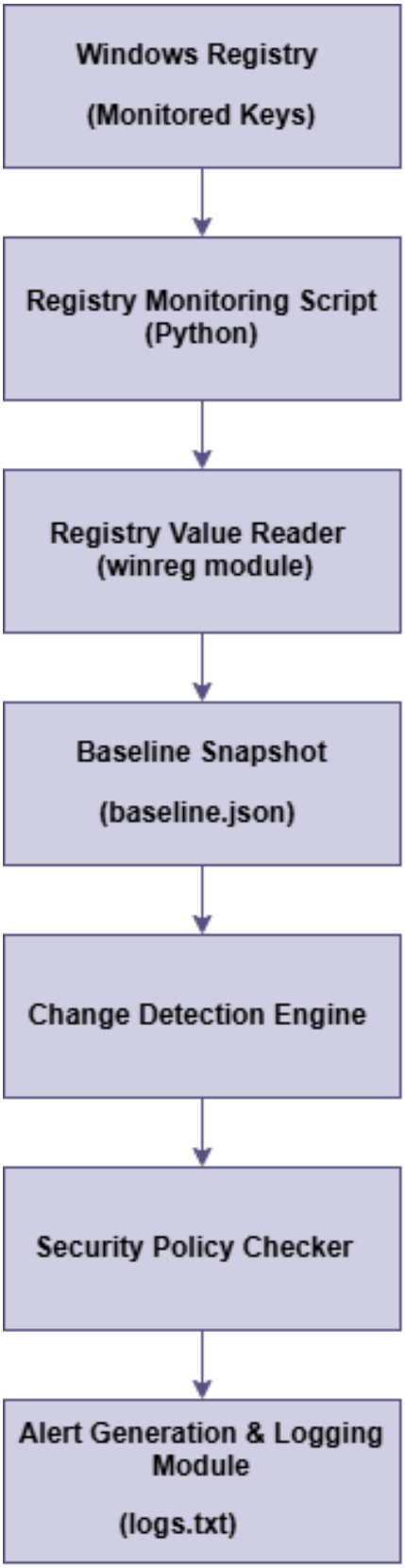

Figure 3: Architecture Diagram of Windows Registry Change Monitoring System

This architecture demonstrates a basic endpoint monitoring mechanism used in cybersecurity environments to detect suspicious system configuration changes.

9. FLOWCHART

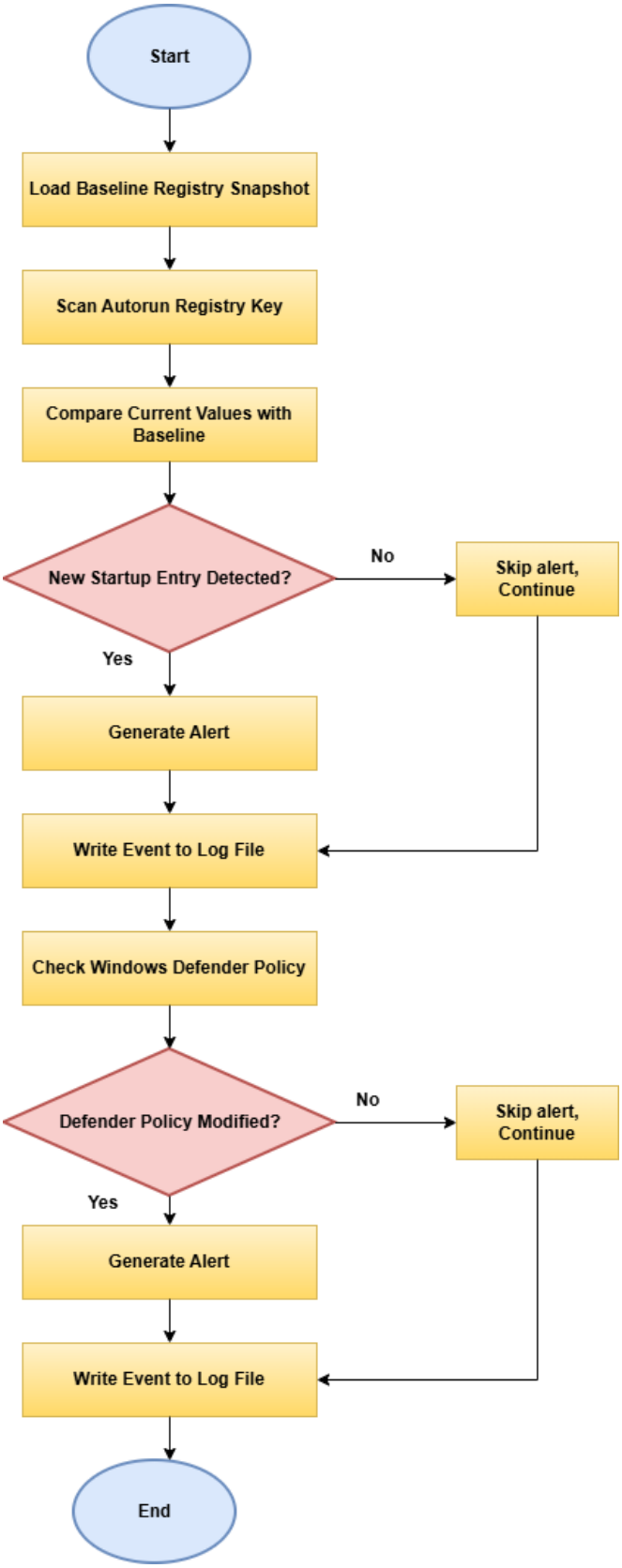

Figure 4: Flowchart

10. EXPECTED PRACTICAL OUTPUT

The Windows Registry Change Monitoring System generates log entries whenever suspicious registry activity is detected during the monitoring process. These logs provide a record of potential security events related to registry modifications.

The monitoring script records all alerts in a log file named **logs.txt**. Each entry includes a timestamp and a description of the detected registry change.

Example Log Entries

2026-01-01 13:22:22 [AUTORUN] New startup entry detected: TestEntry -> C:\test.exe
2026-01-01 13:28:45 [SECURITY] Suspicious Defender policy detected

Each log entry contains the following information:

- **Timestamp** indicating when the registry change was detected
- **Alert category** such as AUTORUN or SECURITY
- **Description of the detected registry modification**

These log entries help identify suspicious system behavior such as unauthorized startup programs or attempts to disable security protections like Windows Defender.

The generated logs can assist security analysts in detecting potential malware persistence mechanisms and investigating system configuration changes.

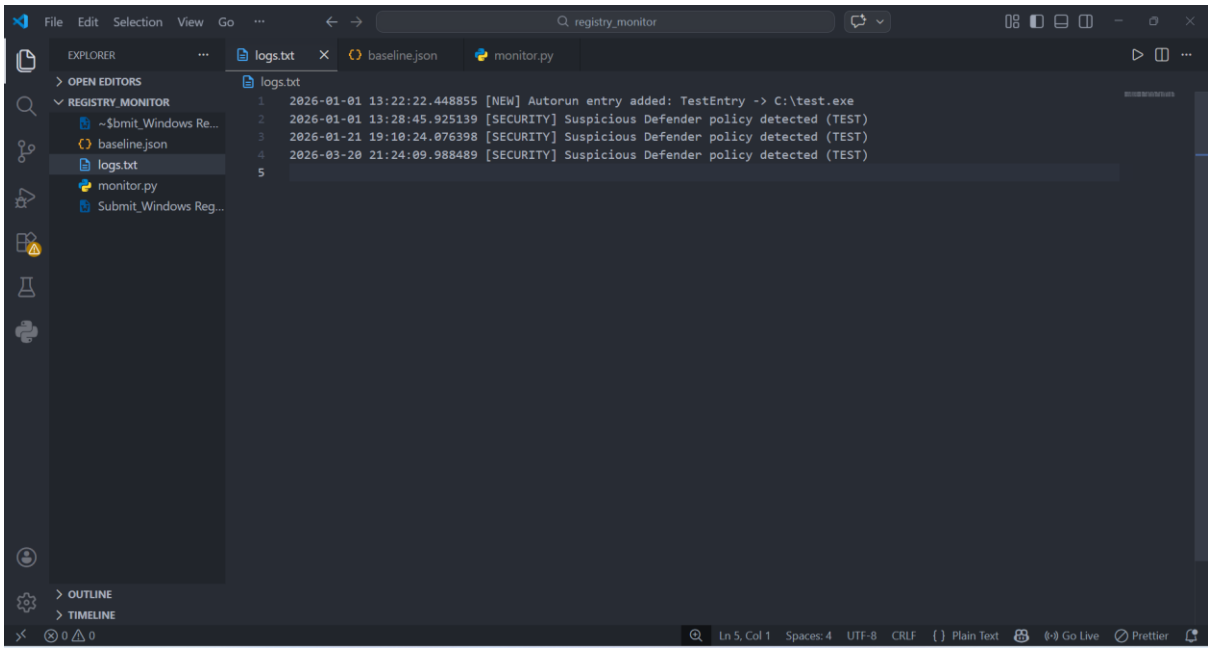

Figure 5: logs.txt

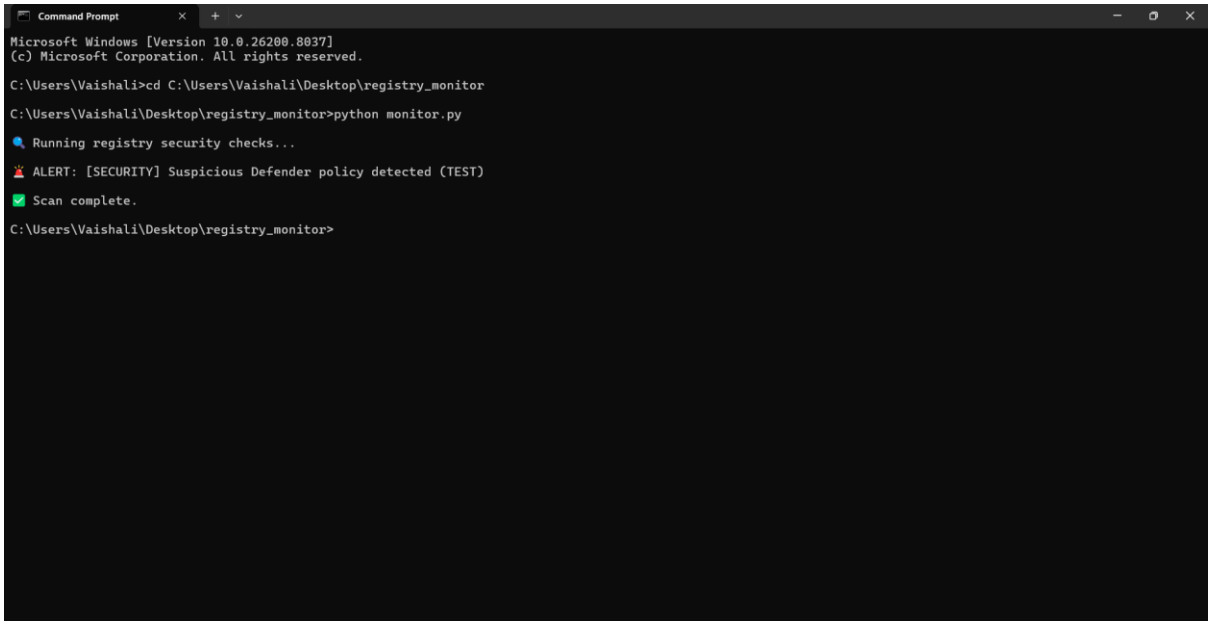

Figure 6: monitor.py running in terminal

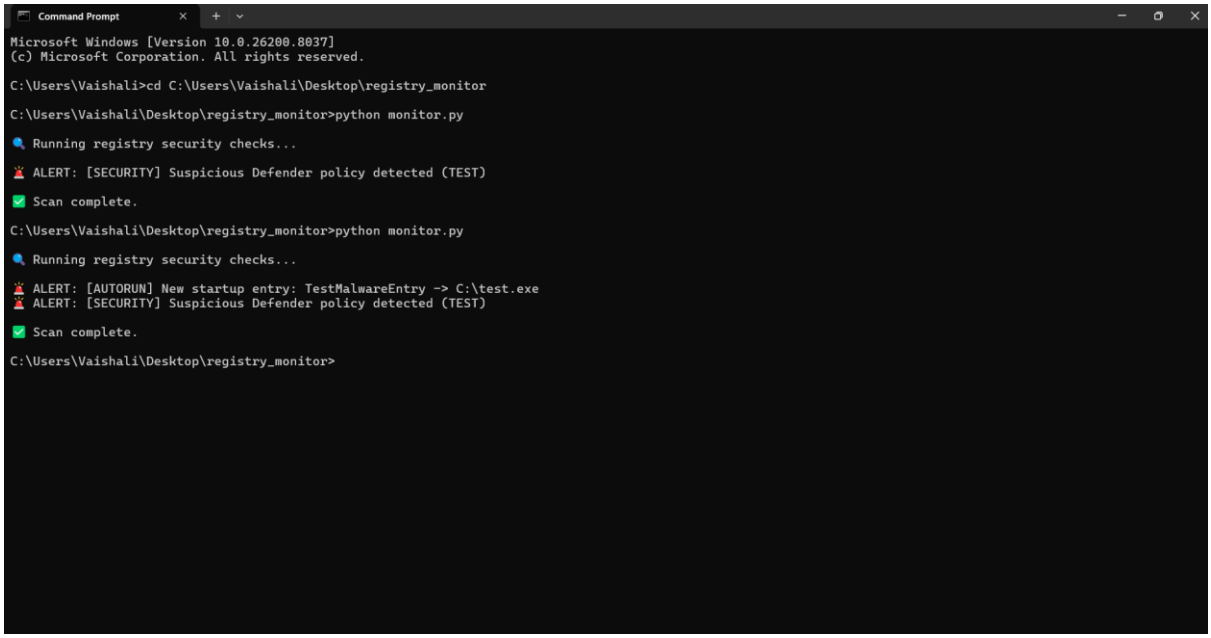

Figure 7: Suspicious Defender policy detected

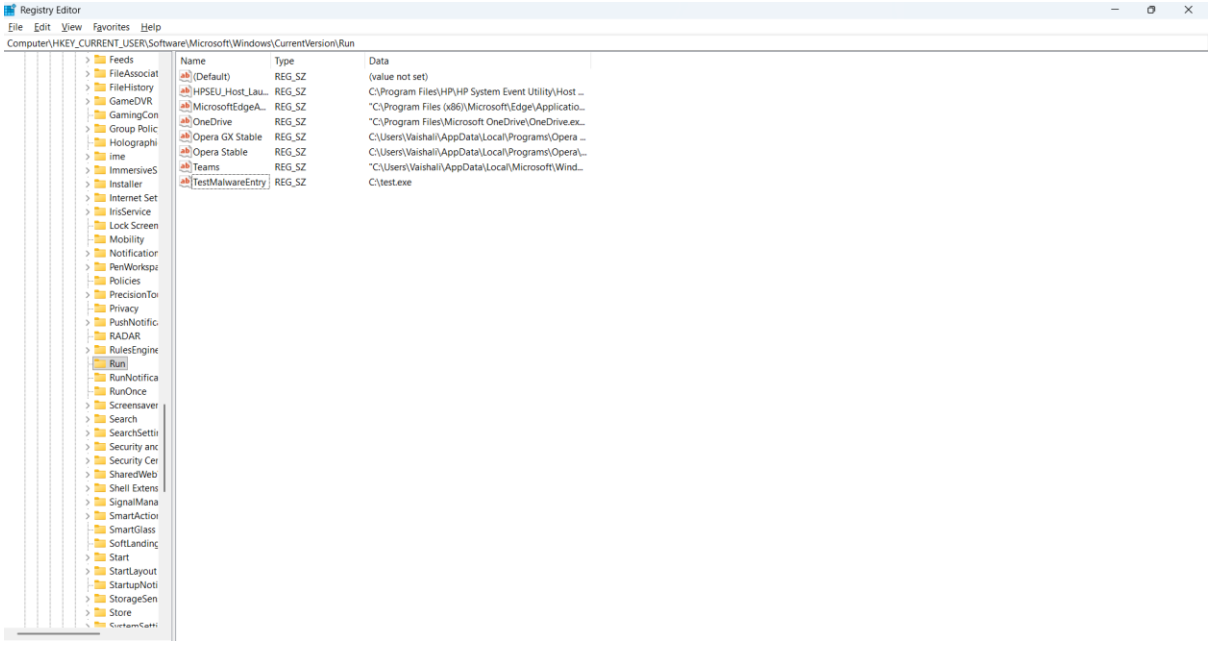

Figure 8: Registry Editor

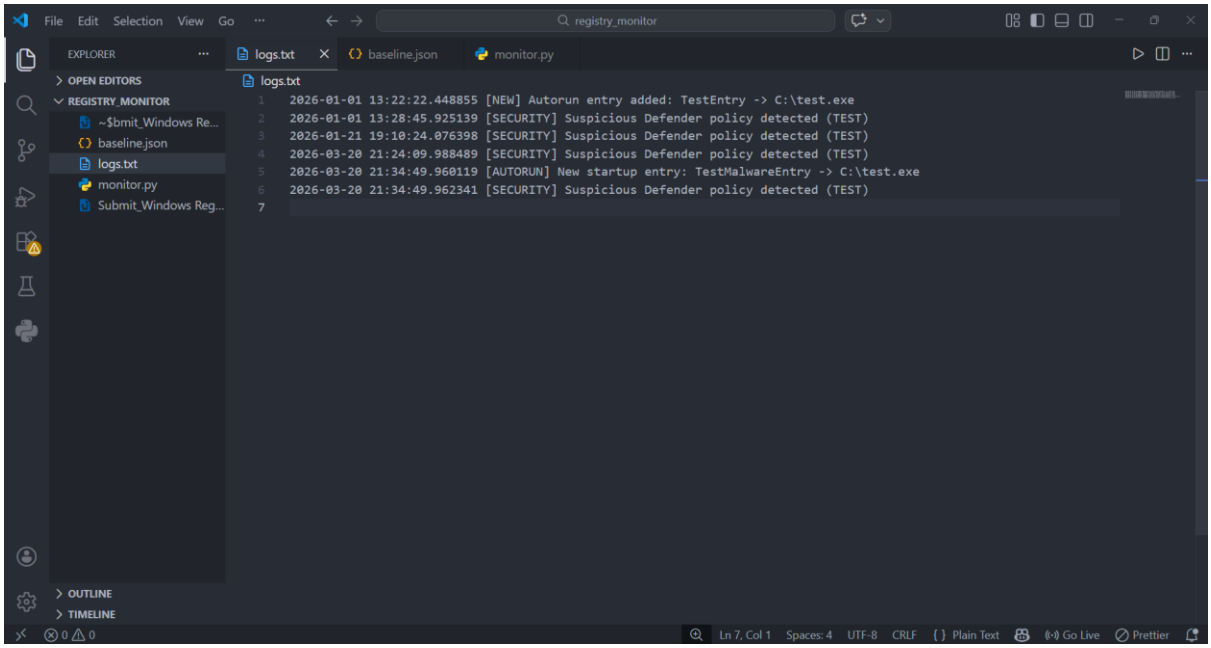

Figure 9: logs.txt

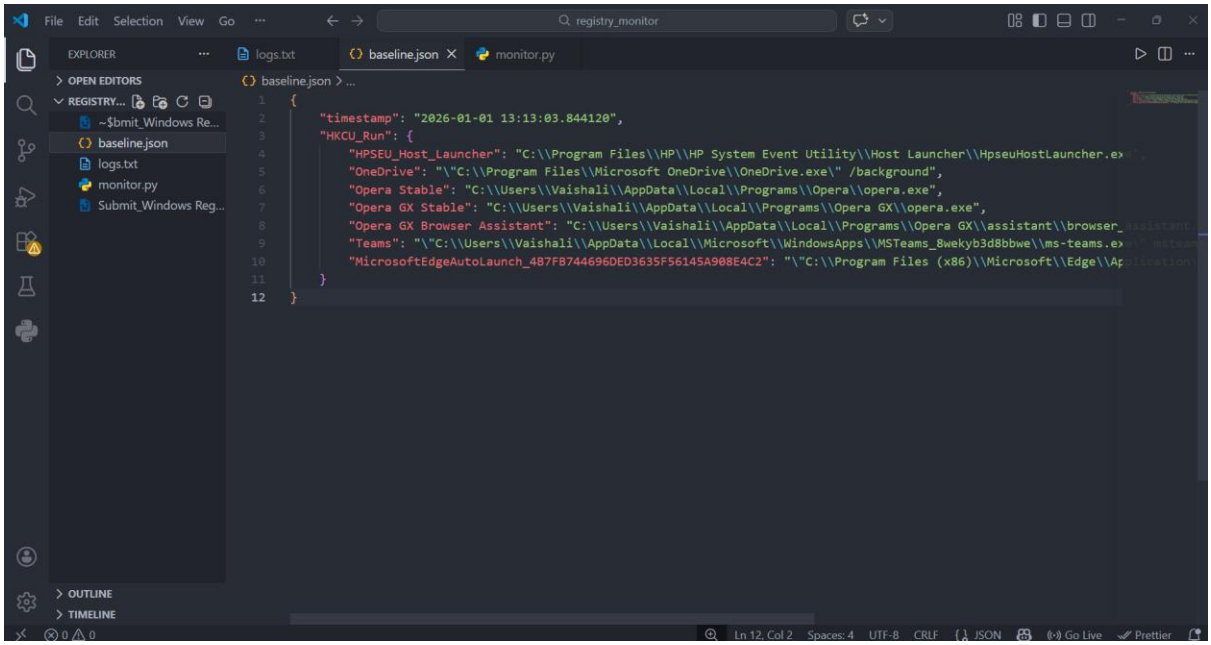

Figure 10: baseline.json

11. LEARNING OUTCOMES

The development of the **Windows Registry Change Monitoring System** provided practical exposure to several important cybersecurity and system monitoring concepts. Through the implementation and testing of this project, the following learning outcomes were achieved:

Understanding the Structure and Role of the Windows Registry

This project helped in understanding how the Windows Registry functions as a central configuration database for the operating system, applications, and security policies. It provided insight into how registry keys and values control system behavior and startup configurations.

Identifying Malware Persistence Techniques

The project demonstrated how malware can establish persistence by adding entries to autorun registry keys. By monitoring these keys, it becomes possible to detect programs that attempt to execute automatically during system startup.

Implementing Registry Monitoring Using Python

The project provided hands-on experience in developing system monitoring scripts using the Python programming language. It involved using the **winreg** module to access and retrieve registry values directly from the Windows operating system.

Detecting Suspicious Configuration Changes

By comparing the current registry state with a baseline snapshot, the project demonstrated how unauthorized or suspicious registry changes can be detected. This technique is commonly used in system integrity monitoring and security auditing.

Logging and Analyzing Security Events

The project also introduced the concept of security event logging. The monitoring system records detected registry changes along with timestamps, allowing security analysts to review logs and investigate potential security incidents.

12. ADVANTAGES OF THE SYSTEM

The **Windows Registry Change Monitoring System** provides several advantages in terms of security monitoring and system integrity verification. The implemented solution demonstrates how registry monitoring can help detect suspicious system behavior and potential security threats.

1. Detection of Malware Persistence Mechanisms

The system monitors the autorun registry key where programs can be configured to start automatically during user login. By detecting newly added startup entries, the system can identify possible malware persistence mechanisms that attempt to execute malicious programs whenever the system starts.

2. Monitoring of Security Policy Modifications

The monitoring system checks registry policy settings related to Windows Defender. If the registry value responsible for disabling Defender is modified, the system generates an alert. This helps detect potential attempts by malware to disable security protections.

3. Baseline-Based Integrity Verification

The project implements a baseline snapshot mechanism using a **baseline.json** file. By comparing the current registry state with the baseline configuration, the system can detect unauthorized additions that were not present during the initial snapshot.

4. Simple and Lightweight Implementation

The monitoring system is implemented using a Python script and built-in modules such as **winreg**, **json**, and **datetime**. Because it does not require complex software dependencies, the system is lightweight and easy to deploy on Windows systems.

5. Security Event Logging for Analysis

Whenever suspicious registry activity is detected, the system generates alerts and records them in a log file. These logs contain timestamps and descriptions of detected changes, which can assist in security monitoring and forensic investigation.

6. Practical Demonstration of Registry Monitoring Techniques

The project demonstrates how registry monitoring can be implemented as part of defensive cybersecurity practices. It provides practical understanding of how security analysts detect unauthorized system configuration changes and investigate potential indicators of compromise.

13. LIMITATIONS OF THE CURRENT SYSTEM

Although the Windows Registry Change Monitoring System demonstrates basic registry monitoring techniques, the current implementation has several limitations that restrict its overall monitoring capability.

1. Limited Autorun Registry Monitoring

The current implementation monitors only one autorun registry key:

HKCU\Software\Microsoft\Windows\CurrentVersion\Run

While this key is commonly used for startup programs, Windows contains multiple other autorun registry locations that can also be used by malware for persistence. Examples include **RunOnce**, **Startup folder entries**, and system-level autorun keys located in **HKLM**. Because the current system monitors only a single autorun location, malicious startup entries placed in other registry paths may not be detected.

2. No Detection of Modified or Deleted Entries

The monitoring system currently focuses only on detecting **newly added registry entries** by comparing the current registry state with the baseline snapshot.

However, the system does not detect:

- Existing registry values that have been **modified**
- Registry entries that have been **deleted**

Attackers may modify existing registry values instead of creating new ones, which can still indicate suspicious behavior. Since the system only checks for new additions, certain types of registry tampering may remain undetected.

3. Single Execution Monitoring

The monitoring script performs only a **single registry scan** when it is executed. After the scan is completed, the program terminates.

This means the system does not continuously monitor the registry for changes over time. If a malicious registry modification occurs after the script has finished executing, the system will not detect it unless the script is manually run again.

Continuous monitoring mechanisms, such as background services or scheduled tasks, are not implemented in the current version.

4. Limited Registry Coverage

The monitoring system inspects only a small number of registry locations related to startup persistence and Windows Defender policies.

The Windows Registry contains thousands of keys and configuration entries that control system behavior. Malware may target many other registry locations to modify system settings, disable security controls, or alter application behavior.

Because the current system monitors only a limited set of registry paths, some malicious registry modifications occurring in other locations may not be detected.

14. FUTURE SCOPE

Although the current implementation provides basic registry monitoring functionality, several improvements can be implemented in future versions of the system to enhance its effectiveness and usability.

1. Continuous Real-Time Monitoring

The current implementation performs a single scan of the registry. Future versions could implement continuous monitoring using scheduled tasks or background services to detect changes in real time.

2. Monitoring Additional Registry Keys

The system currently monitors only a limited set of registry paths. Future improvements could include monitoring additional registry locations frequently targeted by malware, such as:

HKLM\Software\Microsoft\Windows\CurrentVersion\Run
HKLM\Software\Microsoft\Windows\CurrentVersion\RunOnce
Startup-related service registry keys

3. Detection of Modified or Deleted Entries

The current system focuses on detecting newly added registry entries. Future versions could also detect modifications to existing values and deletion of registry entries.

4. Automated Alert Notifications

The system currently logs alerts to a file. Future implementations could include automated alert mechanisms such as:

Email notifications
Desktop notifications
Integration with security monitoring dashboards

5. Integration with Security Monitoring Systems

The registry monitoring system could be integrated with larger security monitoring platforms such as SIEM systems for centralized event analysis.

These enhancements would make the system more robust and closer to real-world endpoint monitoring tools used in cybersecurity environments.

15. CONCLUSION

The **Windows Registry Change Monitoring System** developed in this project demonstrates how basic registry monitoring techniques can be implemented to detect suspicious configuration changes in Windows systems.

The system focuses on monitoring registry locations commonly targeted by malware for persistence mechanisms and security policy tampering. By creating a baseline snapshot and comparing it with the current registry state, the system is able to detect newly added autorun entries that may indicate malicious activity.

In addition, the project includes a security policy check to detect possible attempts to disable Windows Defender through registry modifications.

The monitoring script generates alerts and logs whenever suspicious registry changes are detected. These logs can assist security analysts in identifying potential indicators of compromise and performing further investigation.

Through the development of this project, practical knowledge was gained in areas such as:

Windows Registry structure and functionality
Registry-based malware persistence techniques
Python scripting for system monitoring
Security event logging and analysis

Overall, this project demonstrates a simplified implementation of registry monitoring techniques used in defensive cybersecurity operations and provides a foundation for developing more advanced endpoint monitoring systems.

16. PROJECT DELIVERABLES

The final submission of the **Windows Registry Change Monitoring System** project includes several components that demonstrate the design, implementation, and functionality of the monitoring system. These deliverables provide both the technical implementation and the supporting documentation required for evaluation.

1. Project Documentation

The project documentation provides a detailed explanation of the system design, objectives, workflow, implementation, and results. It describes how the registry monitoring system works, the techniques used for detecting suspicious changes, and the practical learning outcomes of the project.

2. Python Monitoring Script (monitor.py)

The main implementation of the system is provided as a Python script named **monitor.py**. This script performs registry monitoring tasks such as retrieving registry values, comparing them with the baseline configuration, detecting suspicious changes, and generating alerts.

3. Baseline Registry Snapshot (baseline.json)

The **baseline.json** file stores the initial registry snapshot captured during baseline creation. This file acts as a trusted reference that allows the system to compare the current registry state with the original configuration and detect newly added entries.

4. Monitoring Log File (logs.txt)

The system generates a log file named **logs.txt** that records all detected registry changes. Each log entry contains a timestamp, alert category, and description of the detected registry modification. These logs help in monitoring suspicious system activity.

5. System Execution Screenshots

Screenshots are included to demonstrate the practical execution of the monitoring script. These screenshots show the program running in the terminal and the generated log entries that confirm successful detection of registry changes.